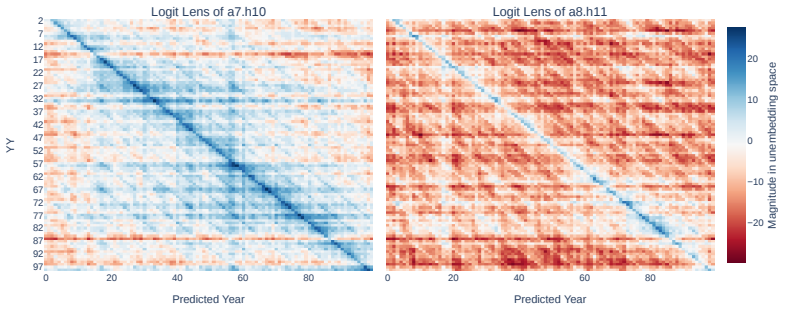

Figure 7: Logit lens of a7.h11 and a8.h10. Axes as in Figure 2; blue indicates that the module upweights the output year, and red, that it downweights the year.

Next, we examine the contributions of attention heads using the logit lens approach [33]: we multiply each head's output by GPT-2's unembedding matrix, translating this output into unembedding (vocabulary) space. Note that here, we do not only view logit lens as a tool for obtaining intermediate estimates of model predictions [4]. Rather, we also use it to understand components' outputs more generally: the logit lens can capture how such outputs shape model predictions, but it can also capture how these outputs add information to the residual stream in unembedding space.

We visualize the heads' outputs for each sentence in our small dataset in Figure 7. Attention head outputs for a sentence with start year `YY` have a high dot product with the embedding vector for `YY`, as shown by the blue diagonal in the plots; this indicates that the head upweights `YY`, making it a more likely output. Note that `YY` was not just upweighted highly compared other 2-digit years; `YY` was the most highly upweighted token across all tokens. Given our earlier analysis, we therefore hypothesize that these attention heads identify the start year (at the `YY` position), and indicate it via a spike in unembedding space of the residual stream at the end position; they thus communicate `YY` to downstream components.

We similarly apply the logit lens to the outputs of MLPs 8-11 (Figure 8). The results indicate that MLPs of 9 and 10 directly specify which years are greater than `YY`: the logit lens of each layer's output has an upper triangular pattern, indicating that they upweight precisely those years greater than `YY`. MLP 11 plays a similar role, but seems to upweight roughly the first 50 years after `YY`, enforcing a maximum duration for the event in the sentence. However, MLP 8 is unusual: its logit lens shows a diagonal pattern, but no upper triangular pattern that would indicate that it computes greater-than.

We claim that this is because MLP 8 contributes mainly indirectly, via the other MLPs in our circuit. We confirm this by patching MLP 8's direct contributions to the logits with the `01`-dataset; we do so again with its indirect contributions, through the other MLPs. In the former case, model performance drops only 14%, while in the latter case, it drops by 39%. So MLP 8 does not contribute much to the logits directly, but it does contribute indirectly. Other MLPs also have mixed effects: MLP 9 has roughly equal direct and indirect contributions (28% vs. 32%), while MLP 10 contributes mostly directly (56% vs. 16%). MLP 11 can only contribute directly.

Our full picture of the circuit so far is this: the attention heads communicate the start year `YY` in embedding space. MLP 8's mostly influences downstream MLPs. However, MLPs 9, 10, and 11 appear to compute the greater-than operation in tandem, and in steps. We conclude that while the attention heads identify the important year `YY`, it is the MLPs that effect the greater-than computation.

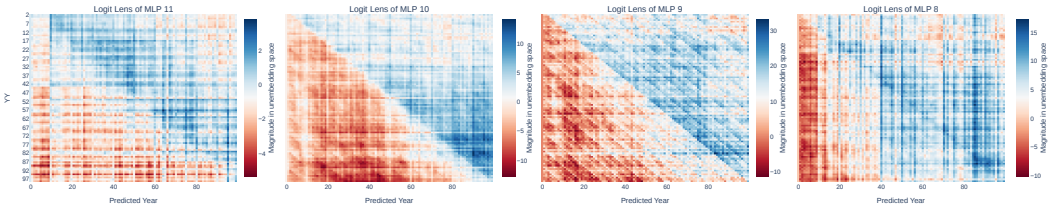

Figure 8: (Left to right) Logit lens of MLPs 11, 10, 9, and 8; labels as in Figure 7

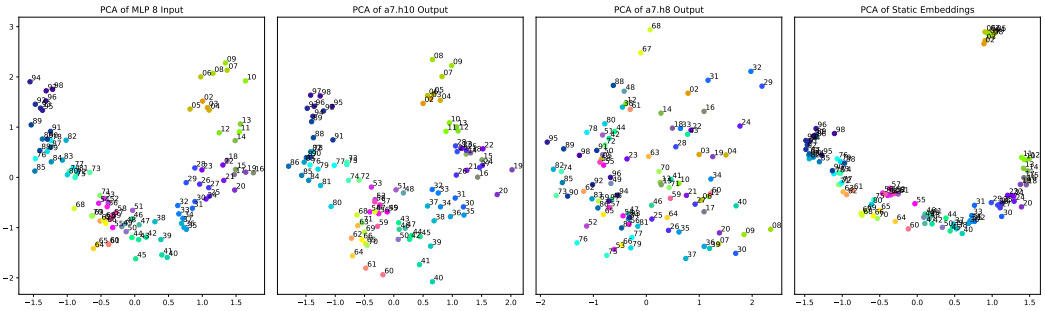

Figure 9: PCA of MLP 8's input, a7.h10's and a7.h8's output, and the static year embeddings. Each point corresponds to one datapoint's representation, and is labeled with and colored by the its YY.

4 Explaining Greater-Than in the Year-Span Prediction Circuit

Our prior experiments show that MLPs 9-11 directly compute greater-than. But how do they do so? We cannot provide a conclusive answer, but identify avenues by which MLPs might compute this. We first examine their inputs, finding structure that might enable greater-than computation. Then, we examine MLP internals, showing how neuron composition could enable greater-than computation.

4.1 Input Structure

To understand how MLPs compute greater-than, we analyze various model representations using 2D Principal Component Analysis (PCA). For each of the 97 datapoints in our small dataset, each with a unique start year, we analyze the input residual stream to our MLPs, as well as the output of relevant attention heads. As a control, we also analyze representations from irrelevant model components, and the static year embeddings. We take all component representations from the end position.

In Figure 9, PCA reveals that the input residual stream to MLP 8 (and indeed all of our MLPs, though not all are shown) is highly structured: representations are ordered by the start year of the sentence they are from, increasing clockwise. The same is true of the outputs of relevant attention heads (a7.h10), which serve as inputs to the MLPs, but not of outputs of irrelevant heads (a7.h8). This suggests that it is specifically the relevant attention heads that transmit this structured information to relevant MLPs. But while the heads seem to transmit this structured information to the MLPs, they need not have created this structure from scratch. We find, as in Wallace et al. [49], that structure already exists in the static year embeddings, though the years 02-09 are clustered apart from the rest. The heads need only unify these groups and move this information from the YY position to the end.

Structured number representations have been implicated in mathematical capability before: Liu et al. [27] train a toy transformer model on modular addition, and find that its number representations become structured only after it stops overfitting and begins to generalize. This suggests that GPT-2's structured number representations may be relevant to its greater-than ability. However, our experiments struggle to prove this causally. When we ablate the dimensions found through PCA, to test their importance to the greater-than task, we found little change in model performance. Similarly, removing linearly-extractable YY information from attention head output using LEACE [5] yielded only slightly lower probability difference (64.7%) than the baseline (81.7%). This indicates that structured YY information may not fully explain GPT-2's greater-than abilities.

4.2 Neuron-Level Processing

To better understand MLPs, we turn to their internals, zooming in on their neurons. We choose to study MLP 10 closely, as we know it directly contributes to the greater-than operation. We start by asking which of MLP 10's neurons are important—a sort of question already studied using probing [10, 12], causal ablations [2, 24, 48], and other techniques; see Sajjad et al. [43] for an overview.

As before, we use path patching because it provides precise causal insights. We path patch each of MLP 10's 3072 neurons' direct contributions to the logits with the 01-dataset. We again record the change in model performance, as measured by probability difference, compared to the unpatched

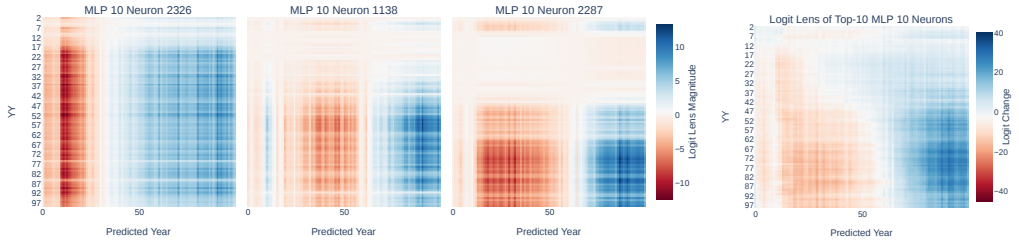

Figure 10: **Left**: The logit lens of the 3 MLP 10 neurons most important to year-span prediction. **Right**: The logit lens of the top-10 MLP 10 neurons. Blue indicates that the neuron upweights logits at the given input year (y-axis), output year (x-axis) combination, while red indicates downweighting.

model. We find that neuron contributions to the task are sparse: most neurons can be patched (ablated) with near zero effect on our model performance, as observed in prior work [48].

We then analyze those neurons that contribute most to model performance using the logit lens. To do this, we take advantage of the fact that each neuron has a corresponding row in the MLP output weight matrix. As noted by Geva et al. [19], multiplying this row by the unembedding weights yields an (unnormalized) distribution over the logits, indicating which outputs the neuron upweights when activated. Taking the outer product of this logit distribution with the neuron's activations yields the logit lens, indicating which output years the neuron upweights for each input sentence's `YY`.

Figure 10 shows the logit lens of the 3 most important neurons in MLP 10; more neurons can be found in Appendix D. Each neuron up- or down-weights certain output years depending on the input year `YY`, but no individual neuron computes greater-than. No one neuron can do so, as each neuron's activation for each input only scales that neuron's distribution over the logits, without changing its overall shape. In contrast, the correct shape of the logits differs depending on the example's start year.

Many neurons can compute greater-than when combined, though. We perform logit lens on the sum of the top-10 neurons' contributions [2] in Figure 10. Though they do not do so individually, the top-10 neurons perform an imperfect greater-than when summed together as a group. The logit lens of MLP 10 as a whole can be thought of as the logit lens of the sum all 3072 neurons' contributions; we partially recreate this with the contributions of just the top-10 neurons. Including more top neurons produces sharper approximations of greater-than; see Appendix D for examples.

In summary, we find that even inside one MLP, the greater-than operation is spread across multiple neurons, whose outputs compose in a sum to form the correct answer. Even the contributions of a small number of relevant neurons composed begin to roughly form the correct operation. We study this in MLP 10, but observe it in the other MLPs as well. Section 3.3 suggested that GPT-2 computed greater-than across multiple important MLPs; these results suggest that moreover multiple important neurons in each MLP compose to allow the MLP to compute greater-than.

5 Does The Circuit Generalize?

We now possess a detailed circuit for year-span prediction. But one thing remains unclear: is this a general circuit for the greater-than operation? Or does it only only apply to this specific, toy task? Answering this question fully would require an in-depth exploration of circuit similarity that fall outside the scope of this paper. For the purposes of this investigation, we perform primarily qualitative analyses of tasks that preserve the format and output space of year-span prediction.

To start, we focus on three increasingly different prompts: "The `<noun>` started in the year 17`YY` and ended in the year 17", "The price of that `<luxury good>` ranges from $ 17`YY` to $ 17", and "1599, 1607, 1633, 1679, 17`YY`, 17". In all cases, a two-digit number greater than `YY` would be a reasonable next token. The model performs greater-than given all of these prompts ($\geq 69\%$ probability difference). Moreover, the circuits found via path patching are similar to those in Sections 3.2 and 3.3. When we run GPT-2 on the first two tasks, ablating all edges not in our original year-span circuit, we achieve 98.8% and 88.9% loss recovery respectively; for more details, see Appendix F. As before,

[2] We can confirm this using path patching as well; see Appendix E for details.